VPSMate

Full MVP Product & Technical Specification

A free, local-first desktop application for managing remote VPS servers.

Document	VPSMate Full MVP Specification
Version	2.0 — React Edition
Updated	8 November 2026
Target platforms	Windows · macOS · Linux
Application model	Locally installed desktop app; direct SSH connections
Cloud dependency	None required
Cost model	Free to install and use; users pay their own VPS provider

This document is the implementation baseline for the VPSMate MVP. It incorporates the confirmed product direction and the updated React-based frontend stack.

1. Product definition

VPSMate is a native desktop application that helps developers and server administrators manage Linux VPS instances through a graphical interface. Users install it on their own computer, add server connection details, and connect directly to their servers over the internet using SSH.

The application runs locally. It is not a hosted web dashboard and does not require a VPSMate cloud account, subscription, hosted database, or mandatory agent installed on the VPS.

The goal is to reduce repeated terminal work while preserving the ability to use a full interactive SSH terminal whenever needed.

Product promise: install VPSMate, connect to your VPS, inspect and manage common server and Laravel tasks from one desktop workspace.

1.1 Confirmed product requirements

- **Desktop only:** deliver installable desktop applications, not a cloud-hosted SaaS product.
- **Cross-platform:** support Windows, macOS, and Linux from one shared codebase.
- **Local-first:** application data and preferences are stored on the user's computer.
- **Direct connection:** connect from the user's machine to the VPS using SSH/SFTP and optional SSH tunnels.
- **Free usage:** no VPSMate subscription, usage-based fee, or account requirement.
- **No required server agent:** use standard Linux tools and SSH capabilities already available on the VPS.
- **Internet required for remote work:** the app can open offline, but connecting to remote VPS servers requires network access.
- **Provider independent:** support any compatible SSH server, including DigitalOcean, AWS, Hetzner, Vultr, and self-managed VPS providers.

1.2 What VPSMate is not

- It is not a cloud control panel that stores the user's server credentials on a VPSMate server.
- It is not a replacement for the VPS provider's billing, snapshots, firewall, or account-management console.
- It does not promise to manage every Linux distribution or every custom service in the first MVP.
- It does not silently run all commands as root.
- It does not require users to expose database ports publicly.

2. Users and primary use cases

The first target users are developers, freelancers, and small teams who manage one or more Linux VPS servers, especially servers running Laravel applications.

2.1 Main user stories

- As a developer, I can save multiple VPS profiles so I do not need to re-enter connection details.
- As a developer, I can connect using an SSH password or private key.
- As a developer, I can see whether a server is reachable and view basic system information.
- As a developer, I can discover installed system services and inspect their status.
- As a Laravel developer, I can find projects and run common Artisan commands in the correct project directory.
- As an administrator, I can open an interactive terminal without leaving the application.
- As a developer, I can browse, upload, and download project files over SFTP.

• As an operator, I can inspect logs and review the history of actions performed in VPSMate.

3. Platforms, installation, and distribution

Platform	Expected deliverables	Notes
Windows	MSI and/or NSIS EXE	Support current 64-bit Windows releases.
macOS	DMG / .app	Build for Apple Silicon and Intel where practical; signing and notarization for public distribution.
Ubuntu / Debian	DEB and AppImage	Test dependencies and desktop integration.
Fedora / compatible RPM distros	RPM and AppImage	Test on supported Fedora releases.
Other Linux	AppImage where supported	State tested distributions clearly.

A single source repository should produce platform-specific builds. Packaging, signing, and notarization are release tasks. macOS signing/notarization may require an Apple Developer account for the publisher, but it does not create a VPSMate subscription for users.

4. Technology stack

Layer	Technology	Responsibility
Desktop shell	Tauri 2	Native window, packaging, OS integration, IPC.
Frontend	React	Application UI and reusable components.
Language	TypeScript	Typed UI, state, API contracts.
Build tool	Vite	Frontend development and production builds.
UI system	shadcn/ui + Tailwind CSS	Accessible components and styling.
State	Zustand	Application and feature state.
Backend	Rust	SSH orchestration, filesystem, local data, secure OS integration.
SSH / SFTP	Rust SSH implementation	SSH sessions, command execution, SFTP, tunnels.
Terminal	xterm.js + PTY bridge	Interactive terminal display and input.
Local database	SQLite	Server metadata, projects, preferences, history.
Credential storage	OS secure credential store	Passwords and private-key passphrases.
Charts	Apache ECharts	Monitoring charts and trends.

The React frontend must not implement SSH itself. It calls typed Tauri commands. Rust owns remote connections, local persistence, security checks, and operating-system integration.

5. Application architecture

```
React UI (pages, components, Zustand stores)
|
| typed Tauri invoke / event APIs
v
Tauri command layer (input validation, authorization, DTOs)
|
v
Rust application services
| | | |
SSH SFTP PTY Local storage
| | | |
+----------+-----------+---------- SQLite / OS keychain
|
v
Remote VPS over SSH
systemd / Laravel / logs / files
monitoring / optional DB tunnels
```

5.1 Architecture rules

- React components do not execute shell commands and do not contain SSH transport logic.
- Frontend feature services wrap Tauri commands and expose typed request/response models.
- Tauri command handlers validate inputs and delegate to Rust services.
- SSH transport code is independent of UI components and business features.
- Use typed errors and structured results. Do not return ambiguous success strings.
- Long-running operations stream progress and output through Tauri events or channels.
- Use separate SSH channels for command execution, terminal sessions, SFTP, and port forwarding as needed.
- Maintain a connection lifecycle per server. Reuse healthy sessions where safe, reconnect deliberately, and close resources cleanly.

6. Main application navigation

Area	Pages / features
Workspace	Overview, Servers, Projects, Monitoring
Server management	Services, Docker & Containers (later), Databases (later), File Manager, Terminal, Logs, Tasks & Cron Jobs (later)
Application	Command History, Security & Credentials, Settings

Keep the first release focused. Later modules may appear as disabled or clearly marked future features only if that does not confuse users. Prefer shipping a smaller complete workflow over a large set of unfinished screens.

7. MVP feature scope

7.1 Must-have features

Feature	MVP behavior
Server profiles	Add, edit, delete, group, connect, disconnect, reconnect, and test a server profile.
SSH authentication	Password and private-key authentication. Support custom SSH port and username.
Host verification	Verify SSH host key on first connection; remember verified fingerprints; stop and warn if a key unexpectedly changes.
Server overview	Show connection state, hostname, OS, kernel, uptime, CPU load, memory, disk, and last update time.
Service manager	Discover systemd units; inspect status; start, stop, and restart supported services; verify the resulting state.
Laravel manager	Detect Laravel projects; run selected Artisan commands from the correct project directory; queue restart; view Laravel logs.
Interactive terminal	PTY-based terminal with input, resize, ANSI colors, tabs or sessions, and disconnect handling.
SFTP file manager	Browse remote directories; upload, download, create folders, rename, and delete with confirmations.
Operation history	Record operation name, server, safe summary, result, time, and duration without recording secrets.
Local security	Store secrets in the OS credential manager/keychain, not in SQLite or logs.

7.2 Should-have if time allows

- Basic log viewer for system and application logs.
- Saved safe command templates.
- Multiple terminal tabs per server.
- SFTP transfer progress, cancellation, and retry.
- Project bookmarks and recent directories.

7.3 Later releases, not required for MVP

- Native PostgreSQL and MySQL user interface through SSH tunnels.
- Docker and container management.
- Cron and scheduled-task editor.
- Advanced historical monitoring, alerts, and notifications.
- Deployment pipelines, backups, snapshots, team sharing, and remote collaboration.
- Cloud sync or hosted account features. These are explicitly outside the current product direction.

8. Server profiles and connection workflow

8.1 Server profile fields

Field	Description
Name	Friendly name, such as Production or Staging.
Host	IP address or DNS hostname.
Port	SSH port; default 22.
Username	SSH account name.
Authentication	Password or private key.
Credential reference	Reference to OS credential storage; never the secret itself.
Group / tags	Optional organization labels.
Notes	Optional non-sensitive notes.

8.2 Connection states

Use explicit states: disconnected, connecting, host-key-verification, connected, reconnecting, and error. Show the current state and useful error detail in the UI.

8.3 First connection

- Resolve the host and establish the SSH handshake.
- Display the host-key fingerprint and ask the user to verify it.
- Persist the fingerprint only after explicit acceptance.
- Authenticate with the selected method.
- Run a small, harmless discovery command such as hostname and uptime.
- Show connection success only after the server responds.

If a previously trusted host key changes, stop the connection and show a security warning. Do not silently accept the new key.

9. Dashboard and monitoring

The MVP monitoring model is lightweight and snapshot-based. It should not require a permanent agent or a high-frequency background process.

Metric	MVP display
Identity	Hostname, distribution, version, kernel, architecture.
Availability	Connected/disconnected state and last successful sample time.
Uptime	System uptime.
CPU	Load average and available CPU information.
Memory	Total, used, available memory and percentage.
Disk	Filesystem mount, total, used, available, percentage.
Network	Basic interface counters where available.

- Display when a metric was last collected.
- Represent stale or unavailable values honestly. Never make old values look live.
- Use reasonable polling intervals and allow refresh on demand.
- Avoid expensive commands or large output for routine snapshots.

10. Service manager

Discover actual services on the target machine instead of assuming every server has the same packages or unit names.

10.1 Common service examples

- Web servers: nginx, apache2, httpd.
- Databases: mysql, mariadb, postgresql.
- Cache and queues: redis, supervisor.
- PHP process managers: php-fpm and versioned PHP-FPM units.
- Other systemd services discovered on the server.

10.2 Service operations

- List discovered units and their current state.
- Show active, inactive, failed, and unknown states distinctly.
- Start, stop, restart, and inspect status.
- Request confirmation for potentially disruptive actions.
- After an action, query the service again and show the verified result.
- Show command output and exit status when an operation fails.

Service control may require sudo. Use a dedicated non-root SSH account with narrowly scoped sudo permissions. Do not assume unrestricted sudo access.

11. Laravel project manager

11.1 Project detection

Allow users to browse to a remote directory or scan selected locations. Identify Laravel projects using indicators such as artisan, composer.json, bootstrap/, and storage/. Do not scan the entire server automatically.

11.2 Project dashboard

- Project name and remote path.
- Detected Laravel and PHP information where safely available.
- Recent operation status.
- Quick links to logs and terminal at the project directory.
- Queue and scheduler information when detectable.

11.3 Safe Artisan actions

Action	Behavior
About	Run php artisan about.
Route list	Run php artisan route:list.
Config cache	Run selected config cache commands with clear labels.
Cache clear	Run selected cache clear commands after user confirmation.
Queue restart	Run php artisan queue:restart and explain that workers restart gracefully.
Migrate status	Run php artisan migrate:status.
Logs	Read Laravel log files with bounded output and safe pagination.

Every operation must use the detected project working directory. Display the command and target path before execution. Database migrations and destructive commands must require explicit confirmation. Do not expose .env secrets in the UI or logs.

11.4 Queue and Supervisor

- Show whether Supervisor is installed and discover relevant program names where practical.
- Show queue worker status from available system information.
- Provide queue:restart as the safe Laravel-level action.
- Do not automatically rewrite Supervisor configuration in the MVP.
- Clearly distinguish a Laravel queue restart from restarting the Supervisor service itself.

12. Interactive terminal

Use xterm.js in React and a Rust-managed PTY/SSH channel. The terminal is a full interactive shell, not a text box that sends one command at a time.

- Support interactive input and streamed output.
- Support ANSI colors and terminal control sequences.
- Send terminal resize events to the remote PTY.
- Support copy, paste, clear, and multiple terminal sessions.
- Handle disconnects, reconnects, and session termination visibly.
- Do not log terminal passwords, private keys, or secret environment values.
- Make it clear which server and user the terminal is connected to.

13. SFTP file manager

- Browse remote folders and files with path navigation.
- Upload and download files and directories where supported.
- Create folders, rename, and delete with confirmation.
- Show file size, permissions, modified time, and transfer progress where available.
- Support cancellation and safe recovery from interrupted transfers.
- Warn before overwriting an existing file.
- Do not silently follow remote symlinks outside the selected path.

SFTP operations should use the authenticated SSH connection and must not require a separate public file-transfer port.

14. Logs and operation history

14.1 Logs

- Provide bounded tail views for common system and Laravel logs.
- Allow refresh and basic text filtering.
- Avoid downloading unbounded log files by default.
- Clearly identify the remote path and time of retrieval.

14.2 Operation history

Record the server, operation category, safe summary, project path if applicable, start time, duration, exit status, and a short sanitized output summary. Do not store passwords, private keys, passphrases, tokens, or raw environment files.

15. Database access direction

A native database browser is not a required MVP feature. When implemented later, connect to PostgreSQL or MySQL through an SSH local port forward. Do not ask users to expose database ports to the public internet just for VPSMate.

- Create a local-only tunnel bound to loopback by default.
- Allow the user to configure database host and port as seen from the VPS.
- Close tunnels when the connection or application session ends.
- Never store database passwords in plaintext.

16. Security and privacy

16.1 Local-first data handling

- SQLite stores metadata and references, not raw credentials.
- Store passwords, private-key passphrases, and other secrets in the platform secure credential store.
- Do not send server inventory, credentials, logs, or project contents to a VPSMate cloud service.
- Do not add telemetry by default. If optional diagnostics are ever introduced, make them opt-in and explain exactly what is sent.
- Provide a way to remove a server profile and its associated local credential reference.

16.2 SSH and command safety

- Verify SSH host keys and detect unexpected changes.
- Use least-privilege SSH accounts and scoped sudo rules.
- Use structured command requests and fixed command templates where possible.
- Never build shell commands by concatenating untrusted user input.
- Validate paths and arguments. Quote or encode them safely for the remote execution mechanism.
- Show destructive or high-impact actions and require confirmation.
- Capture exit code, stdout, stderr, duration, and cancellation state.
- Redact known secret patterns from diagnostic output and logs.

16.3 Sensitive operations

Examples requiring extra confirmation include deleting remote files, overwriting files, stopping production services, restarting critical services, running database migrations, and executing destructive Artisan commands. The confirmation should identify the server, target path or service, and operation.

17. Local database design

Suggested initial SQLite schema. Use migrations and stable IDs. Add indexes for server and history lookups.

Table	Key fields
servers	id, name, host, port, username, auth_type, credential_ref, group_name, created_at, updated_at
known_hosts	id, server_id, key_type, fingerprint, first_seen_at, verified_at
projects	id, server_id, name, remote_path, project_type, detected_at
saved_commands	id, name, category, command_template, requires_confirmation
operation_history	id, server_id, project_id, operation, safe_summary, exit_code, started_at, duration_ms
preferences	key, value, updated_at
port_forwards	id, server_id, bind_host, local_port, remote_host, remote_port, created_at

Do not store secrets directly in any SQLite column. Store only a credential reference that points to the operating system's secure storage.

18. Suggested repository structure

```
vpsmate/
■■■ src/
■ ■■■ app/
■ ■■■ components/
■ ■ ■■■ ui/
■ ■ ■■■ layout/
■ ■ ■■■ shared/
■ ■■■ features/
■ ■ ■■■ servers/
■ ■ ■■■ dashboard/
■ ■ ■■■ monitoring/
■ ■ ■■■ services/
■ ■ ■■■ projects/
■ ■ ■■■ terminal/
■ ■ ■■■ files/
■ ■ ■■■ logs/
■ ■ ■■■ settings/
■ ■■■ stores/
■ ■■■ hooks/
■ ■■■ services/ # typed Tauri wrappers
■ ■■■ types/
■ ■■■ main.tsx
■■■ src-tauri/
■ ■■■ src/
■ ■ ■■■ main.rs
■ ■ ■■■ commands/
■ ■ ■■■ core/
■ ■ ■■■ ssh/
■ ■ ■■■ sftp/
■ ■ ■■■ terminal/
■ ■ ■■■ discovery/
■ ■ ■■■ services/
■ ■ ■■■ laravel/
■ ■ ■■■ monitoring/
■ ■ ■■■ security/
■ ■ ■■■ storage/
■ ■■■ capabilities/
■ ■■■ Cargo.toml
■■■ migrations/
■■■ tests/
■ ■■■ integration/
■ ■■■ fixtures/
■■■ package.json
■■■ README.md
```

19. Frontend organization

Use feature-based React organization. Keep pages thin and move server behavior into typed service modules and Rust commands.

- Use React function components and TypeScript types for all feature contracts.
- Use Zustand for shared application state such as server list, active connection, and UI preferences.
- Use local component state for short-lived form and dialog state.
- Keep Tauri invoke calls inside service wrappers, not scattered across UI components.
- Use React hooks for subscriptions to Tauri events and clean up listeners on unmount.
- Use a consistent loading, empty, success, and error state for every remote operation.
- Use accessible keyboard behavior and clear focus states.

20. UX and visual direction

- Desktop-first layout with a compact left navigation sidebar.
- Server switcher that makes the active server obvious.

- Connection status always visible when a server is selected.
- Use a clean, modern interface with readable typography and restrained color.
- Use dark and light themes.
- Use tables for services, files, and operation history where appropriate.
- Use confirmation dialogs for destructive operations.
- Show command output in a terminal-like panel with copy support.
- Avoid showing a metric without its unit, sample time, or unavailable state.

20.1 Suggested main layout

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ VPSMate Settings Profile■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Overview ■ Active server / page title ■
■ Servers ■■■■■■■■■■■■■■■■■■■
```

22. Testing strategy

Test area	Required coverage
Rust unit tests	Command construction, validation, parsing, redaction, state transitions.
Frontend tests	Forms, status display, stores, loading/error states, confirmation dialogs.
SSH integration	Password/key auth, host verification, disconnect, reconnect, timeouts.
Service tests	Discovery and status changes on supported systemd distributions.
Laravel tests	Project detection, working directory, Artisan output and failure handling.
PTY tests	Input/output, resize, ANSI output, session cleanup.
SFTP tests	Browse, upload, download, overwrite, cancel, interrupted transfer.
Security tests	No plaintext secrets in SQLite, logs, crash reports, or diagnostics.
Platform tests	Install, launch, update strategy, and uninstall on Windows, macOS, and Linux.

23. Development milestones

Milestone	Deliverable	Exit condition
1. Foundation	Tauri 2 + React + TypeScript + Vite, navigation, design system, SQLite migrations.	App launches locally on developer platform.
2. SSH foundation	Profiles, password/key auth, host verification, lifecycle, reconnect.	Connect and run hostname/uptime safely.
3. Discovery	OS information and basic monitoring snapshots.	Accurate server overview with sample timestamps.
4. Services	systemd discovery and service actions.	Actions verified after execution.
5. Laravel	Project detection, Artisan tools, queue restart, logs.	Commands run in correct project directory.
6. Terminal + SFTP	Interactive PTY and remote file manager.	Usable terminal and reliable file transfers.
7. Hardening	History, security review, packaging, docs, cross-platform tests.	Release candidates for all target platforms.

24. First development vertical slice

Do not start by building every screen. First prove that the desktop app can establish a secure SSH connection and report a real result.

- Create the Tauri 2 + React + TypeScript + Vite project.
- Build a simple Add Server screen.
- Accept host, port, username, and one authentication method.
- Connect and display the SSH host-key fingerprint for verification.
- After verification and authentication, run hostname and uptime.
- Display stdout, stderr, exit code, and connection state.
- Disconnect cleanly and remove temporary session resources.

This vertical slice tests the highest-risk part of the product before investing in the rest of the UI.

25. MVP acceptance criteria

- A user can install and launch VPSMate locally on supported Windows, macOS, and Linux builds.
- No VPSMate cloud account or hosted backend is required.
- A user can add multiple server profiles.
- SSH password and private-key authentication work.
- Host keys are verified and unexpected changes block connection.
- The app can connect, disconnect, and reconnect with clear status.
- The dashboard displays real server information with timestamps.
- Systemd services are discovered rather than assumed, and service actions are verified.
- Laravel projects are detected and supported Artisan commands run in the correct directory.
- The interactive terminal supports PTY input, output, and resize.
- SFTP supports browsing, upload, and download.
- Secrets are not stored in plaintext SQLite or application logs.
- Errors and permissions failures are reported honestly.
- The application does not require an agent or publicly exposed database ports.

26. Risks and decisions

Risk / decision	Mitigation
Different Linux distributions	Start with Ubuntu/Debian and Fedora systemd; discover commands and service units at runtime.
SSH library behavior across OSes	Prototype connection, host-key verification, PTY, SFTP, and key formats early.
Sudo permissions differ	Document least-privilege setup and report permission failures clearly.
Long-running commands	Stream output, support cancellation where safe, and avoid blocking the UI.
Remote shell injection	Use structured operations, strict validation, safe argument handling, and fixed templates.
macOS distribution	Plan signing/notarization and test Apple Silicon and Intel builds.
Local credential storage differences	Use native OS credential APIs and test on all supported platforms.

27. Explicit out-of-scope items

- Hosted VPSMate web dashboard.
- Mandatory VPSMate account, cloud sync, or subscription.
- Mandatory remote agent or daemon.
- Automatic deployment pipeline or CI/CD platform.
- Full cloud-provider control plane for billing, snapshots, and firewall management.
- Unrestricted root execution as the default.
- Native database administration UI in the first MVP.
- Advanced alerting and team collaboration in the first MVP.

28. Product definition of done

VPSMate MVP is ready when a developer can install the desktop application on a supported operating system, add a VPS, securely connect over SSH, inspect the server, manage discovered services, work with a Laravel project, open an interactive terminal, transfer files, and review operation history without relying on a VPSMate cloud service.

The product should remain useful on a laptop with no VPSMate account and no special server-side agent. The only remote requirement is a reachable SSH server and the permissions needed for the operations the user chooses to perform.

VPSMate • Full MVP Technical Specification • Version 2.0 • React Edition • 8 November 2026